

ClaimCheck — Final Build Roadmap

3 People, 12 Real Hours, Equal Work Split

THE PRODUCT IN 30 SECONDS

Upload a startup pitch deck PDF → tool reads it → finds every technical claim → Googles each one → scores them 0-100 with evidence → shows a dashboard with green/amber/red verdicts.

THE FOUR STATIONS (how it works under the hood)

- Station 1 — THE READER:** Opens the PDF, extracts all text. (Library does the work, 30 min)
- Station 2 — THE SPOTTER:** Claude reads the text and picks out the verifiable claims, ignoring marketing fluff. (Claude prompt, 2h to get right)
- Station 3 — THE INVESTIGATOR:** For each claim, runs 3 Google searches via Serper.dev, collects results, labels each source by reliability tier. (Search API + Python, 2h)
- Station 4 — THE JUDGE:** Claude reads each claim + the evidence found, says what supports/contradicts it, flags red flags. Then a simple maths formula turns that into a consistent 0-100 score. (Claude prompt + formula, 2h)
- THE FRONTEND:** Upload button → loading screen → results dashboard with scores and expandable details. (Lovable, 3-4h)
-

TEAM ROLES

Person	Role	Focus
Person A	Backend Dev 1	Station 1 (Reader) + Station 2 (Spotter) + API server
Person B	Backend Dev 2	Station 3 (Investigator) + Station 4 (Judge) + PDF export
Person C	Frontend + Product + Demo	Frontend in Lovable + prompt testing + demo PDFs + pitch

Everyone has roughly equal work. A and B can work in parallel from the start because their stations are independent.

BEFORE THE HACKATHON (Friday evening)

Person A:

- ☐ Create GitHub repo with this folder structure:

```
claimcheck/
├── backend/
│   ├── main.py          # API server (Person A)
│   ├── pdf_extractor.py  # Station 1 (Person A)
│   ├── claim_extractor.py # Station 2 (Person A)
│   ├── evidence_retriever.py # Station 3 (Person B)
│   ├── scorer.py        # Station 4 (Person B)
│   └── prompts/
│       ├── extraction.txt # Spotter prompt
│       └── scoring.txt    # Judge prompt
├── frontend/            # Person C via Lovable
├── demo/
│   ├── sample_deck_1.pdf
│   ├── sample_deck_2.pdf
│   └── cached_results.json # Backup demo data
└── .env                 # API keys
```

- ☐ Install Python dependencies: `pip install anthropic requests fastapi uvicorn pymupdf python-multipart`
- ☐ Test a basic Claude API call works

Person B:

- ☐ Sign up at serper.dev → get free API key (2 min)
- ☐ Install same Python dependencies
- ☐ Test a basic Serper.dev search call works
- ☐ Pull the GitHub repo

Person C:

- ☐ Sign up for Lovable (you're getting free Pro at the hackathon)
- ☐ Find 2-3 real startup websites with bold technical claims (YC companies are good hunting ground)
- ☐ Create 1-2 fake pitch deck PDFs in Google Slides with a mix of real and exaggerated claims, export as PDF
- ☐ Paste one claim into ChatGPT, screenshot the messy paragraph response (for demo comparison)
- ☐ Pull the GitHub repo

Everyone:

- ☐ Save the Anthropic credits link from the hackathon email (claim Saturday morning)
- ☐ Read this entire document so you know what you're building and why

SATURDAY — BUILD DAY

9:00-11:00 | PANELS + SETUP (everyone in the room)

You're in the opening panels and partner intros. Use gaps to set up.

Person A:

- Claim Anthropic API credits
- Create `.env` file with API keys
- Push to GitHub so everyone has it
- Open your editor, have `pdf_extractor.py` ready to go

Person B:

- Claim Anthropic credits
- Pull repo, confirm `.env` works
- Open `evidence_retriever.py`, ready to go
- Test one Serper.dev API call from your laptop to make sure it works on the venue WiFi

Person C:

- Claim Anthropic credits + Lovable Pro access code
- Open Lovable
- Start describing the UI (see Frontend section below for exact description)
- Have your demo PDFs ready on your laptop

MILESTONE: API keys work. Everyone has the repo. Lovable is open. Ready to build at 11:00.

11:00-12:30 | FIRST BUILD SPRINT (1.5h)

Person A — Station 1 (Reader) + Start Station 2 (Spotter)

Station 1 — PDF text extraction (30 min):

Build `pdf_extractor.py`. This opens a PDF and pulls out all the text.

What it does in plain English: someone gives you a PDF file → you give back all the words in it as plain text.

```
python
```

```
import pymupdf

def extract_text_from_pdf(file_path: str) -> str:
    doc = pymupdf.open(file_path)
    full_text = ""
    for page in doc:
        full_text += page.get_text() + "\n"
    doc.close()
    return full_text
```

Test it with one of Person C's demo PDFs. Print the first 500 characters. Does it look right?

Station 2 — Claim extraction prompt (start, finish after lunch):

Build `claim_extractor.py`. This sends the PDF text to Claude and asks it to find the technical claims.

What it does in plain English: you give Claude a wall of text from a pitch deck → Claude gives you back a clean list of the 3-7 specific claims worth fact-checking, plus suggested Google searches for each one.

```
python
```

```
import anthropic
```

```
import json
```

```
import os
```

```
client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
```

```
EXTRACTION_PROMPT = """You are a senior technical due diligence analyst at a top VC firm.
```

I'm going to give you text extracted from a startup's pitch deck.

YOUR JOB: Find every verifiable technical claim in this text.

WHAT COUNTS AS A VERIFIABLE CLAIM:

- Any specific number: "95% accuracy", "10x faster", "300 tokens per second"
- Any comparison to a named competitor or baseline: "outperforms GPT-4", "faster than H100"
- Any "first to market" or "only solution" assertion
- Any reference to a benchmark, dataset, or published result
- Any specific cost, scale, or efficiency claim

WHAT TO IGNORE:

- Vague marketing: "world-class team", "cutting-edge technology", "revolutionary approach"
- Opinions or future projections: "we believe the market will..."
- Team credentials, funding amounts, or business model claims

For each verifiable claim you find, return:

1. "original_text" — the exact quote from the deck
2. "claim_type" — one of: performance_benchmark, speed_comparison, accuracy_claim, market_position, cost_claim, scale_claim
3. "key_entities" — the specific models, products, benchmarks, or numbers mentioned
4. "search_queries" — exactly 3 Google search queries that would help verify this claim. Make them specific and targeted.

IMPORTANT: Return ONLY valid JSON, no markdown, no explanation. Just the JSON:

```
{
  "claims": [
    {
      "id": 1,
      "original_text": "...",
      "claim_type": "...",
      "key_entities": ["..."],
      "search_queries": ["...", "...", "..."]
    }
  ],
  "ignored_statements": ["list of marketing statements you correctly ignored"]
}
```

```
def extract_claims(text: str) -> dict:
```

```
    response = client.messages.create(
```

```

model="claude-sonnet-4-20250514",
max_tokens=4000,
messages=[{
    "role": "user",
    "content": f"{EXTRACTION_PROMPT}\n\n--- PITCH DECK TEXT ---\n{text}"
}]
)

response_text = response.content[0].text

# Clean markdown code blocks if Claude adds them
if "```json" in response_text:
    response_text = response_text.split("```json")[1].split("```")[0]
elif "```" in response_text:
    response_text = response_text.split("```")[1].split("```")[0]

return json.loads(response_text.strip())

```

Spend the remaining time testing this prompt. Paste in your demo pitch deck text and check:

- Does it find the right claims?
- Does it ignore the marketing fluff?
- Are the search queries specific enough?

Iterate on the prompt wording until it works consistently.

Person B — Station 3 (Investigator)

Build `evidence_retriever.py`. This takes a claim's search queries and Googles them.

What it does in plain English: for each claim, you run 3 Google searches and collect the results. You also label each result by how reliable the source is (academic paper = very reliable, Reddit = not reliable).

```
python
```

```
import requests
```

```
import os
```

```
SERPAPI_KEY = os.getenv("SERPAPI_KEY")
```

```
# How reliable is each source? Tier 1 = most reliable, Tier 4 = least
```

```
SOURCE_TIERS = {
```

```
    # Tier 1: Academic and independent benchmarks
```

```
    "arxiv.org": 1,
```

```
    "paperswithcode.com": 1,
```

```
    "huggingface.co": 1,
```

```
    "mlperf.org": 1,
```

```
    "nature.com": 1,
```

```
    "ieee.org": 1,
```

```
    "openreview.net": 1,
```

```
    # Tier 2: Reputable tech press and company research blogs
```

```
    "techcrunch.com": 2,
```

```
    "reuters.com": 2,
```

```
    "theverge.com": 2,
```

```
    "semianalysis.com": 2,
```

```
    "venturebeat.com": 2,
```

```
    "wired.com": 2,
```

```
    "arstechnica.com": 2,
```

```
    "openai.com": 2,
```

```
    "anthropic.com": 2,
```

```
    "research.google": 2,
```

```
    # Tier 3: Blogs, company pages, unknown
```

```
    "medium.com": 3,
```

```
    "substack.com": 3,
```

```
    "towardsdatascience.com": 3,
```

```
    # Tier 4: Social media, forums
```

```
    "reddit.com": 4,
```

```
    "twitter.com": 4,
```

```
    "x.com": 4,
```

```
    "quora.com": 4,
```

```
}
```

```
def classify_source(url: str) -> int:
```

```
    """What tier is this source? Returns 1-4. Default 3 (unknown)."""
```

```
    for domain, tier in SOURCE_TIERS.items():
```

```
        if domain in url.lower():
```

```
            return tier
```

```
    return 3
```

```
def search_web(query: str, num_results: int = 5) -> list:
```

```
    """Run one Google search via Serper.dev. Returns list of results."""
```

```

response = requests.post(
    "https://google.serper.dev/search",
    json={"q": query, "num": num_results},
    headers={"X-API-KEY": SERPER_API_KEY}
)

if response.status_code != 200:
    print(f"Search failed for: {query}")
    return []

results = response.json().get("organic", [])

return [
    {
        "title": r.get("title", ""),
        "snippet": r.get("snippet", ""),
        "url": r.get("link", ""),
        "source_tier": classify_source(r.get("link", "")),
        "source_domain": r.get("link", "").split("/")[2] if len(r.get("link", "").split("/")) > 2 else "unknown"
    }
    for r in results
]

def retrieve_evidence(claim: dict) -> list:
    """For one claim, run all its search queries and combine results. Remove duplicates."""
    all_evidence = []
    seen_urls = set()

    for query in claim.get("search_queries", []):
        results = search_web(query)
        for r in results:
            if r["url"] not in seen_urls:
                seen_urls.add(r["url"])
                all_evidence.append(r)

    # Sort: most reliable sources first
    all_evidence.sort(key=lambda x: x["source_tier"])

    return all_evidence

```

Test it independently with hardcoded claims — you don't need to wait for Person A:

```
python
```

```

test_claim = {
    "search_queries": [
        "Groq LPU inference speed benchmark",
        "Groq vs Nvidia H100 tokens per second",
        "Llama 2 70B inference performance comparison"
    ]
}
results = retrieve_evidence(test_claim)
for r in results:
    print(f"[Tier {r['source_tier']}] {r['source_domain']}: {r['title']}")

```

Check: Are you getting real results? Are the tiers assigned correctly? Are there duplicates?

Person C — Frontend (Upload Page + Results Dashboard)

Open Lovable and describe the UI. Here's what to tell it:

For the upload page: "Build me a single-page web app called ClaimCheck. At the top, a heading that says 'ClaimCheck' with a subtitle 'AI-Powered Technical Due Diligence'. Below that, a large drag-and-drop area where a user can upload a PDF file. Below that, a text area with placeholder text 'Or paste pitch deck text here'. Below that, a button that says 'Analyze Claims'. Use a clean, professional design with a dark blue and white colour scheme."

For the loading state: "When the user clicks Analyze, show a loading screen. Display a progress indicator with status messages that update: 'Reading document...', 'Found X claims...', 'Checking evidence...', 'Scoring claims...'. Make it feel like real work is happening."

For the results dashboard: "After loading, show results. At the top, a summary bar: 'Analyzed X claims: Y verified, Z uncertain, W unverified. Average credibility: N/100'. Below that, a list of claim cards. Each card shows: a coloured circle (green for score 70+, amber for 40-69, red for 0-39), the score as a large number, the original claim text, and a short summary. Each card is clickable and expands to show: Evidence For (green section with source links), Evidence Against (red section with source links), Red Flags (amber badges), and Follow-up Questions. At the bottom, an 'Export PDF' button."

Build the whole thing with dummy data first (use the hardcoded JSON from below). You can connect it to the real backend later.

Dummy data to build with:

```

json

```

```
{
  "summary": {
    "total_claims": 3,
    "verified": 1,
    "uncertain": 1,
    "unverified": 1,
    "average_score": 59
  },
  "claims": [
    {
      "id": 1,
      "original_text": "Our LPU delivers Llama 2 70B inference at 300 tokens/sec, 10x faster than Nvidia H100",
      "score": 87,
      "verdict": "green",
      "summary": "Claim is well-supported. Independent benchmarks confirm ~300 tok/s for Groq LPU vs 30-40 tok/s for H100",
      "evidence_for": [
        {"text": "SemiAnalysis benchmark shows Groq LPU at 300 tok/s on Llama 2 70B", "source_url": "https://semianalysis.com/2024/03/26/groq-lpu-benchmark-300-tok-s-on-llama-2-70b/", "source_domain": "semianalysis.com"},
        {"text": "Artificial Analysis confirms Groq leading single-query inference speed", "source_url": "https://artificialanalysis.ai/groq-lpu-benchmark-300-tok-s-on-llama-2-70b/", "source_domain": "artificialanalysis.ai"}
      ],
      "evidence_against": [
        {"text": "At batch sizes >1, Groq advantage drops to 3-5x vs H100", "source_url": "https://arxiv.org/abs/2403.12777v1", "source_domain": "arxiv.org"}
      ],
      "red_flags": ["Batch size not specified in claim — advantage varies significantly with batch size"],
      "follow_up_questions": ["What batch size was used in your benchmark?", "FP16 or BF16 precision?", "What is latency vs throughput?"]
    },
    {
      "id": 2,
      "original_text": "Achieves 95% accuracy on LongBench, surpassing GPT-4o by 12 points",
      "score": 38,
      "verdict": "red",
      "summary": "LongBench has been superseded by LongBench v2 since 2024. GPT-4o scores 50.1% on LongBench v2, making the claim outdated.",
      "evidence_for": [],
      "evidence_against": [
        {"text": "LongBench v2 released 2024, original version considered outdated", "source_url": "https://paperswithcode.com/sota/longbench-on-llm-on-2024-03-26", "source_domain": "paperswithcode.com"},
        {"text": "GPT-4o scores 50.1% on LongBench v2", "source_url": "https://huggingface.co/blog/gpt4o", "source_domain": "huggingface.co"}
      ],
      "red_flags": ["Uses deprecated benchmark version", "No published paper or evaluation methodology", "No public repository for evaluation code"],
      "follow_up_questions": ["Which version of LongBench was used?", "Can you share the evaluation code and methodology?"]
    },
    {
      "id": 3,
      "original_text": "First AI-powered solution for real-time structural health monitoring in offshore wind turbines",
      "score": 52,
      "verdict": "amber",
      "summary": "Several companies (Akselos, Sensorium, Restrata) offer AI-based structural monitoring for wind turbines. The claim is partially supported but lacks specific evidence.",
      "evidence_for": [
        {"text": "Akselos offers AI-based structural health monitoring for wind turbines", "source_url": "https://www.akselos.com/solutions/offshore-wind-turbines", "source_domain": "akselos.com"},
        {"text": "Sensorium offers AI-based structural health monitoring for wind turbines", "source_url": "https://www.sensorium.ai/solutions/offshore-wind-turbines", "source_domain": "sensorium.ai"},
        {"text": "Restrata offers AI-based structural health monitoring for wind turbines", "source_url": "https://www.restrata.com/solutions/offshore-wind-turbines", "source_domain": "restrata.com"}
      ],
      "evidence_against": [
        {"text": "No specific evidence provided for the claim", "source_url": null, "source_domain": null}
      ],
      "red_flags": ["Lacks specific evidence for the claim"],
      "follow_up_questions": ["Which AI solution was used?", "What specific metrics were used to measure performance?"]
    }
  ]
}
```

```

    {"text": "Limited academic literature on real-time AI monitoring specifically for offshore (vs onshore) turbines", "source": "https://www.researchgate.net/publication/354123456"},
    "evidence_against": [
      {"text": "Akselos offers AI-driven structural monitoring for offshore structures since 2021", "source_url": "https://techcrunch.com/2021/08/12/akselos-ai-structural-monitoring-offshore-structures/"},
      {"text": "Multiple startups in AI structural health monitoring space identified", "source_url": "https://venturebeat.com/2021/07/28/multiple-startups-in-ai-structural-health-monitoring-space-identified/"},
    ],
    "red_flags": ["'First' claim contradicted by existing competitors", "No patent found for the specific approach"],
    "follow_up_questions": ["How do you differentiate from Akselos and Sensorium?", "What specifically is 'real-time' — what does it mean?"]
  }
]
}

```

Also during this time:

- Test Person A's Spotter prompt when it's ready — give feedback from a VC perspective
- Make sure demo PDFs have a good mix of claim types

MILESTONE by 12:30: Person A can extract claims from a PDF. Person B can search Google for evidence. Person C has a frontend that looks good with dummy data. Three independent pieces, all working.

12:30-13:30 | LUNCH

Eat. Talk through any issues. Don't code — you need the mental break.

13:30-15:30 | SECOND BUILD SPRINT (2h)

Person A — Finish Station 2 + Build API Server

Finish Station 2: Keep iterating on the claim extraction prompt based on Person C's feedback. Make sure it works on at least 2 different demo PDFs.

Build API Server (`main.py`): This is the glue that connects everything. One endpoint that accepts a PDF, runs it through all 4 stations, and returns the results as JSON.

```
python
```

```
from fastapi import FastAPI, UploadFile, File, Form
from fastapi.middleware.cors import CORSMiddleware
from typing import Optional
import tempfile
import os

from pdf_extractor import extract_text_from_pdf
from claim_extractor import extract_claims
from evidence_retriever import retrieve_evidence
from scorer import score_claim

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.post("/analyze")
async def analyze(
    file: Optional[UploadFile] = File(None),
    text: Optional[str] = Form(None)
):
    # STATION 1: Get text
    if file:
        with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp:
            content = await file.read()
            tmp.write(content)
            tmp_path = tmp.name
            input_text = extract_text_from_pdf(tmp_path)
            os.unlink(tmp_path)
    elif text:
        input_text = text
    else:
        return {"error": "Provide a PDF or text"}

    # STATION 2: Find claims
    extraction = extract_claims(input_text)
    claims = extraction.get("claims", [])

    # STATION 3 + 4: For each claim, get evidence and score it
    results = []
    for claim in claims:
        evidence = retrieve_evidence(claim)
```

```
scored = score_claim(claim, evidence)
results.append(scored)

# Summary stats
scores = [r["score"] for r in results]
avg = sum(scores) // len(scores) if scores else 0

return {
    "summary": {
        "total_claims": len(results),
        "verified": sum(1 for r in results if r["verdict"] == "green"),
        "uncertain": sum(1 for r in results if r["verdict"] == "amber"),
        "unverified": sum(1 for r in results if r["verdict"] == "red"),
        "average_score": avg
    },
    "claims": results
}

# Run: uvicorn main:app --reload --port 8000
```

Test it:

```
bash

curl -X POST http://localhost:8000/analyze -F "file=@demo/sample_deck_1.pdf"
```

Person B — Station 4 (Judge) + PDF Export

Station 4 — Scoring system (`scorer.py`): Build the Judge. This is two things combined: Claude reading the evidence and giving a qualitative assessment, plus a maths formula giving a consistent numerical score.

What it does in plain English: Claude reads the claim and all the Google results. It decides what supports the claim, what contradicts it, what red flags exist, and what questions to ask. Then a simple formula counts up the evidence and produces a number.

```
python
```

```
import anthropic
```

```
import json
```

```
import os
```

```
client = anthropic.Anthropic(api_key=os.getenv("ANTHROPIC_API_KEY"))
```

```
SCORING_PROMPT = """You are a technical due diligence analyst at a VC firm.
```

```
CLAIM FROM STARTUP PITCH DECK:
```

```
{claim_text}
```

```
EVIDENCE FOUND FROM WEB SEARCH:
```

```
{evidence_text}
```

```
Assess this claim's credibility based on the evidence. Return ONLY valid JSON:
```

```
{{  
  "summary": "2-3 sentence assessment. Be specific — cite numbers from the evidence.",  
  "evidence_for": [  
    {{ "text": "what supports the claim, citing specific data", "source_url": "URL" }}  
  ],  
  "evidence_against": [  
    {{ "text": "what contradicts the claim, citing specific data", "source_url": "URL" }}  
  ],  
  "red_flags": ["specific red flags like: no peer review, deprecated benchmark, cherry-picked metric, no public repo, superlatives"],  
  "follow_up_questions": ["2-3 specific questions a VC should ask the founder"]  
}}
```

```
RULES:
```

- Only cite evidence that actually appears in the search results above
- Do NOT make up or hallucinate any sources
- Be specific with numbers — don't say "the claim seems exaggerated", say "evidence shows X while they claim Y"
- If there's insufficient evidence, say so clearly"""

```
def score_claim_with_claude(claim: dict, evidence: list) -> dict:
```

```
    """Get Claude's qualitative assessment of a claim."""
```

```
    # Format evidence into readable text for Claude
```

```
    evidence_text = ""
```

```
    for i, e in enumerate(evidence[:10]): # Limit to top 10 results
```

```
        evidence_text += f"\n[Source {i+1}] {e['source_domain']} (Reliability: Tier {e['source_tier']})\n"
```

```
        evidence_text += f"Title: {e['title']}\n"
```

```
        evidence_text += f"Snippet: {e['snippet']}\n"
```

```
        evidence_text += f"URL: {e['url']}\n"
```

```
    if not evidence_text.strip():
```

```
        evidence_text = "NO EVIDENCE FOUND — no relevant search results returned."
```

```

prompt = SCORING_PROMPT.format(
    claim_text=claim["original_text"],
    evidence_text=evidence_text
)

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=2000,
    messages=[{"role": "user", "content": prompt}]
)

response_text = response.content[0].text
if "```json" in response_text:
    response_text = response_text.split("```json")[1].split("```")[0]
elif "```" in response_text:
    response_text = response_text.split("```")[1].split("```")[0]

return json.loads(response_text.strip())

```

```

def calculate_score(assessment: dict, evidence: list) -> int:
    """Deterministic formula. Same input = same score. Always.

```

This is what makes us different from 'just pasting into ChatGPT'.

The formula is transparent and consistent.

```

score = 50 # Start neutral

```

```

# Count supporting evidence by tier

```

```

for e in assessment.get("evidence_for", []):

```

```

    url = e.get("source_url", "")

```

```

    tier = 3 # default

```

```

    for ev in evidence:

```

```

        if ev["url"] == url:

```

```

            tier = ev["source_tier"]

```

```

            break

```

```

    if tier == 1: score += 15 # Academic/benchmark = strong support

```

```

    elif tier == 2: score += 10 # Reputable press = moderate support

```

```

    elif tier == 3: score += 3 # Company blog = weak support

```

```

    elif tier == 4: score += 1 # Social media = barely counts

```

```

# Count contradicting evidence by tier

```

```

for e in assessment.get("evidence_against", []):

```

```

    url = e.get("source_url", "")

```

```

    tier = 3

```

```

for ev in evidence:
    if ev["url"] == url:
        tier = ev["source_tier"]
        break

if tier == 1: score -= 20 # Academic contradiction = strong hit
elif tier == 2: score -= 12 # Press contradiction = moderate hit
elif tier == 3: score -= 5 # Blog contradiction = minor hit
elif tier == 4: score -= 2 # Social media = barely counts

# Red flags penalty
score -= len(assessment.get("red_flags", [])) * 5

# No evidence found at all = suspicious
if not assessment.get("evidence_for") and not assessment.get("evidence_against"):
    score -= 15

return max(0, min(100, score))

def get_verdict(score: int) -> str:
    if score >= 70: return "green"
    elif score >= 40: return "amber"
    else: return "red"

def score_claim(claim: dict, evidence: list) -> dict:
    """Full scoring pipeline: Claude assessment + deterministic formula."""

    # Claude reads evidence and gives qualitative assessment
    assessment = score_claim_with_claude(claim, evidence)

    # Formula produces consistent numerical score
    numerical_score = calculate_score(assessment, evidence)

    # Traffic light
    verdict = get_verdict(numerical_score)

    return {
        "id": claim.get("id"),
        "original_text": claim.get("original_text", ""),
        "claim_type": claim.get("claim_type", ""),
        "score": numerical_score,
        "verdict": verdict,
        "summary": assessment.get("summary", ""),
        "evidence_for": assessment.get("evidence_for", []),
        "evidence_against": assessment.get("evidence_against", []),
    }

```

```
"red_flags": assessment.get("red_flags", []),  
"follow_up_questions": assessment.get("follow_up_questions", [])  
}
```

Test with a hardcoded claim + evidence (don't need the full pipeline yet):

```
python  
  
test_claim = {"id": 1, "original_text": "Our model is 10x faster than H100"}  
test_evidence = [  
    {"title": "Groq benchmark", "snippet": "Groq achieves 300 tok/s...", "url": "https://semianalysis.com/test", "source_tier": 2,  
]  
result = score_claim(test_claim, test_evidence)  
print(f"Score: {result['score']}, Verdict: {result['verdict']}")
```

PDF Export (last 30 min of this sprint):

Simplest approach — generate HTML and convert to PDF:

```
python
```

Add to main.py or a new file

from fastapi.responses import HTMLResponse

def generate_report_html(results: dict) -> str:

html = f"""

<html><head><style>

body {{ font-family: Arial; padding: 40px; }}

.green {{ color: #16a34a; }} .amber {{ color: #d97706; }} .red {{ color: #dc2626; }}

.claim {{ border: 1px solid #ddd; padding: 16px; margin: 12px 0; border-radius: 8px; }}

.score {{ font-size: 28px; font-weight: bold; }}

</style></head><body>

<h1>ClaimCheck Report</h1>

<p>Claims analyzed: {results['summary']['total_claims']} |

Average credibility: {results['summary']['average_score']}/100</p>

<hr>

"""

for claim in results["claims"]:

color = claim["verdict"]

html += f"""

<div class="claim">

{claim['score']}/100

<p>Claim: "{claim['original_text']}"</p>

<p>Assessment: {claim['summary']}</p>

<p>Red flags: {' '.join(claim['red_flags']) if claim['red_flags'] else 'None'}</p>

</div>

"""

html += "</body></html>"

return html

The frontend can open this HTML in a new tab and the user hits Ctrl+P to save as PDF. Good enough for a demo.

Person C — Connect Frontend + Test Pipeline

Connect Lovable frontend to real backend:

Once Person A has the API server running, update your Lovable frontend to call it:

javascript

```
const analyzeDocument = async (file) => {
  const formData = new FormData();
  formData.append('file', file);

  const response = await fetch('http://localhost:8000/analyze', {
    method: 'POST',
    body: formData,
  });

  return await response.json();
};
```

If Person A isn't ready yet, keep building with dummy data — you'll swap in the real API call later.

Test the pipeline end-to-end:

- Upload a demo PDF through the frontend
- Check the results: Do the claims make sense? Are the scores reasonable?
- Feed back to Person A (prompt issues) and Person B (scoring issues)

MILESTONE by 15:30: The full pipeline works. PDF in browser → real scored results out. May be rough, but functional.

15:30-17:30 | INTEGRATION + FIRST POLISH (2h)

Person A — Integration bugs + error handling

- Fix any bugs that appeared when connecting all 4 stations together
- Add basic error handling:
 - What if the PDF has no text? → Return "Could not extract text from this PDF"
 - What if Claude returns malformed JSON? → Wrap in try/except, retry once
 - What if Serper fails? → Return "Search unavailable" for that claim, still score based on what you have
- Test with Person C's demo PDFs: do all 2-3 produce good results?

Person B — Help fix integration + caching

- Help Person A debug integration issues
- **Build the caching system:** Run each demo PDF through the pipeline, save the JSON results to `demo/cached_results.json`. Add an endpoint:

```
@app.get("/demo/{demo_id}")
async def get_demo_results(demo_id: int):
    """Return pre-cached results for demo. Backup if live API fails on stage."""
    with open(f"demo/cached_result_{demo_id}.json") as f:
        return json.load(f)
```

- This is your safety net. If anything breaks on stage, Person C switches to cached results and the audience never knows.

Person C — Frontend polish + start demo prep

- Polish the loading animation — make the status messages update smoothly
- Make sure the expanded claim view looks clean
- Add the "Export Report" button (opens the HTML report from Person B's code)
- **Start the ChatGPT comparison:** Take one of your demo claims, paste it into ChatGPT right now, screenshot the response. You'll show this side-by-side with your tool's output during the pitch.

MILESTONE by 17:30: Full product works in browser. Frontend is polished. Cached results ready as backup.

17:30-18:30 | DINNER

Eat. Don't skip this. You need energy for the final push.

18:30-22:00 | FINAL SPRINT (3.5h)

Person A (1.5h work, then help C)

- Final bug fixes
- Run each demo PDF 3 times — are results consistent? If scores vary wildly between runs, tweak the prompt or formula
- Make sure the server starts cleanly with one command: `uvicorn main:app --port 8000`
- After backend is solid, help Person C rehearse the demo

Person B (1.5h work, then help C)

- Final testing: try edge cases — very long PDF, PDF with zero claims, PDF with only images
- Make sure cached results load instantly
- After backend is solid, help Person C with pitch slides

Person C — Demo prep + pitch

Build pitch slides (5 max):

Slide 1: **The Problem** "95% of early-stage investors never verify startup technical claims. The ones who do spend 4+ hours per claim Googling manually. Technical due diligence is 70% of DD time and the highest risk area."

Slide 2: **Live Demo** (No slide — you switch to the live product and demo it)

Slide 3: **How It Works** Simple diagram: PDF → Claim Extraction → Evidence Search → Scoring → Report
Mention: "Standardised scoring rubric — same claim always gets the same score. Full audit trail."

Slide 4: **Why Not Just Use ChatGPT?** Side-by-side screenshot: ChatGPT paragraph vs your scored table
"ChatGPT gives you a paragraph. We give you a repeatable, auditable, exportable system."

Slide 5: **Team + Market** "Built by VC analysts who do this manually every week. VCs spend £X per deal on due diligence. We're replacing the 4-hour Google session with a 30-second scan."

Write the exact demo script:

[Slide 1 on screen]

"Every week, VCs review dozens of pitch decks. Most contain bold technical claims — '10x faster', '95% accurate', 'first to market'.
95% of early-stage investors never verify these claims.
Not because they don't care — because it takes 4 hours per claim.
We built ClaimCheck."

[Switch to live product]

"Here's a real pitch deck."

[Drag PDF onto upload zone]

"ClaimCheck reads 14 pages and finds the technical claims automatically."

[Claims appear]

"5 verifiable claims found. Now it's checking each one against real sources — arXiv, benchmark leaderboards, academic papers."

[Results load]

"Claim 1 — 87 out of 100, green. The speed claim checks out, with a caveat about batch size."

"Claim 2 — 38 out of 100, red. They cite a deprecated benchmark. No paper published."

[Click to expand red claim]

"Look — specific evidence, red flags, and follow-up questions ready for your next founder meeting."

[Slide 4 — ChatGPT comparison]

"Here's the same claim pasted into ChatGPT."

[Point to paragraph] "A hedge, no score, no sources."

[Point to your tool] "A scored verdict with evidence and an audit trail."

[Slide 5]

"We built the tool we wish we had."

ClaimCheck — technical due diligence in 30 seconds, not 4 hours."

Rehearse this 3 times with the live product. Time it. If it's over 3 minutes, cut something.

MILESTONE by 22:00: Product works, demo is rehearsed, pitch is tight, cached results are ready as backup. Go home and sleep.

SUNDAY 9:00-14:00

9:00-10:00 — Final checks (Everyone)

- Pull latest code, make sure everything still works
- Run demo end-to-end one more time
- Check: does it work on venue WiFi?
- Have Person A ready to hotspot from phone if WiFi is flaky

10:00-12:00 — Polish + edge cases (A + B) / Rehearse (C)

- A + B: Fix any overnight issues, test on the projector screen if possible
- C: Rehearse the pitch 2 more times. Rehearse switching from slides to live demo smoothly.

12:00-13:00 — Lunch + final rehearsal

- Eat
- One final full run-through as a team
- Decide: who presents? (Probably Person C / Vasilis — you know the VC angle best)

13:00-14:00 — Buffer

- Be ready at your laptop 15 min before judging
- Have two tabs open: live product + pitch slides
- Have cached results JSON ready in case you need to switch
- Deep breaths

14:00 — Judging begins

TOTAL WORK PER PERSON

Person	Hours of Active Work	What They Built
Person A	~10h	PDF extraction, claim extraction prompt, API server, integration, bug fixes
Person B	~10h	Web search pipeline, source tiers, scoring system, PDF export, caching, edge cases
Person C	~10h	Frontend in Lovable, prompt testing/feedback, demo PDFs, pitch slides, demo script, ChatGPT comparison, rehearsal

Roughly equal. All three are critical — without any one person, the demo doesn't work.

API KEYS NEEDED

Service	Purpose	Cost	Sign up
Anthropic (Claude)	Brain of the product	\$50 free at hackathon	Credits link in email

Service	Purpose	Cost	Sign up
Serper.dev	Google search results	Free (2,500 searches)	serper.dev
Lovable	Frontend builder	Free Pro at hackathon	Access code at event

That's it. Three accounts total.

INSTALL COMMANDS

```
bash

# Backend (Person A + B)
pip install anthropic requests fastapi uvicorn pymupdf python-multipart

# Run backend
cd backend
uvicorn main:app --reload --port 8000

# Frontend (Person C via Lovable — no install needed, it's a web tool)
# Or if using React:
npx create-next-app@latest frontend --typescript --tailwind
cd frontend && npm run dev
```

IF EVERYTHING GOES WRONG

Cut these to save time:

- PDF upload → paste text only (saves 30 min)
- Loading animations → just a spinner (saves 30 min)
- PDF export → skip, just show dashboard (saves 1h)
- Multiple demo PDFs → just one that works perfectly (saves 1h)

Never cut these:

- The claim extraction prompt (Station 2) — this IS the product
- The scoring system (Station 4) — this is what differentiates you from ChatGPT
- The ChatGPT side-by-side comparison — this is what wins the pitch
- Rehearsing the demo — a stumbled demo loses, a smooth demo wins

DONE. GO BUILD IT.